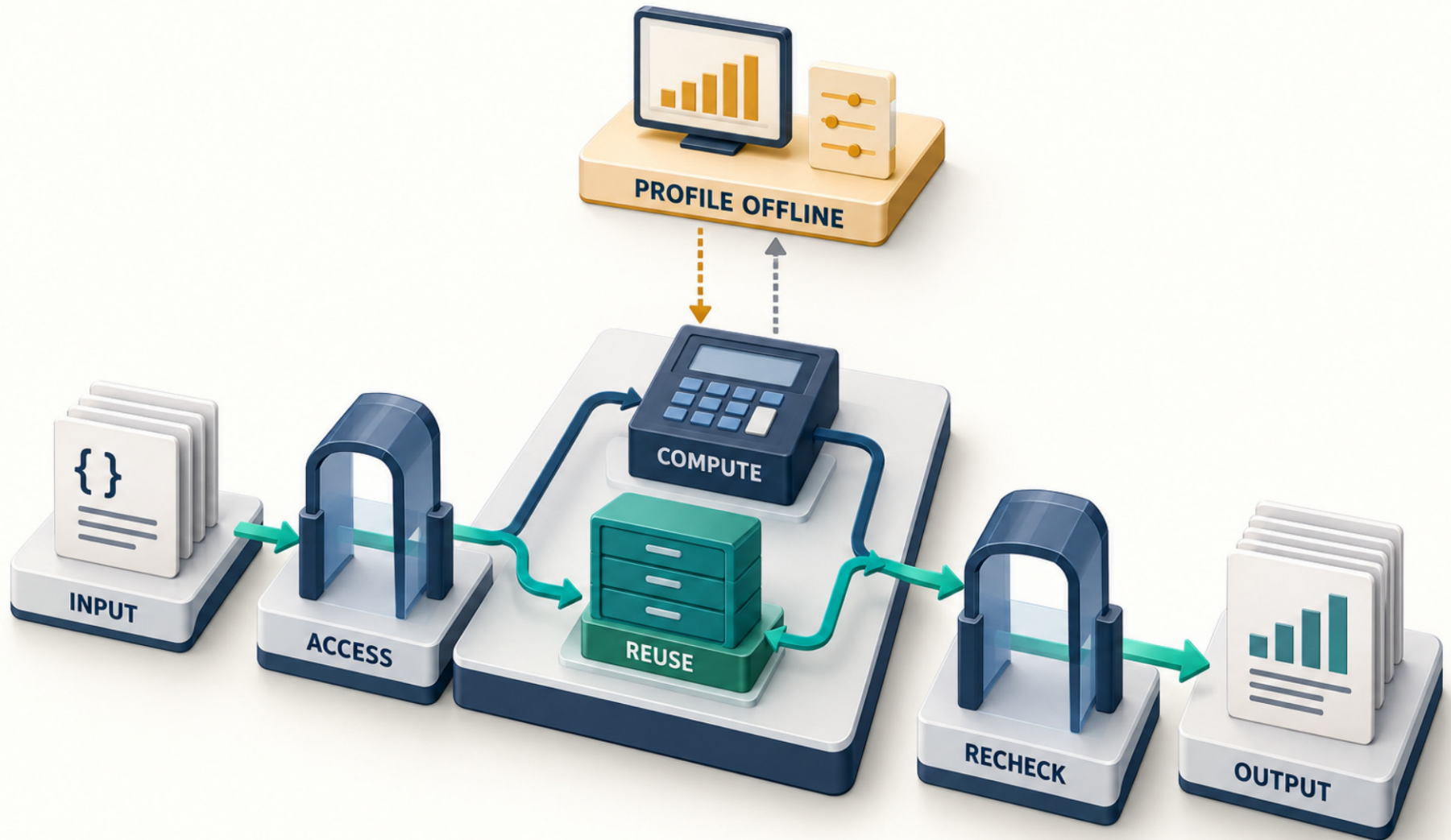


Thetabase

Reuse useful work. Recheck access. Adapt between runs.



3D view of the software design. Runtime traces inform a fixed configuration for later runs.

What gets optimized

Start with complete-result caching. Add reusable stages only where the measurements justify them.

BASELINE / Whole-job reuse

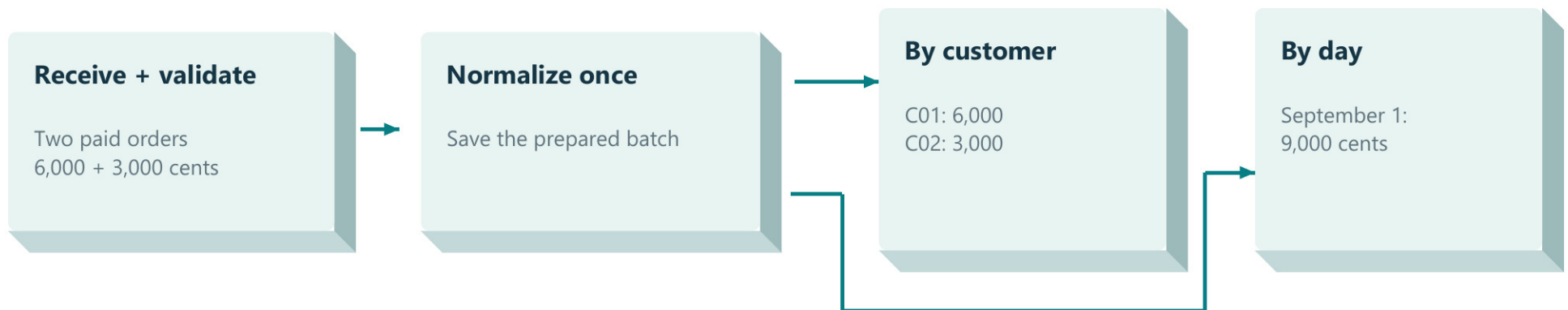
The same complete request can reuse a saved result. A changed request runs the pipeline again.

EXTENSION / Stage reuse

A new question can reuse unchanged preparation, then compute only the new report.

One batch, two reports

Synthetic order data / exact integer arithmetic



Reuse the prepared batch for the second view

BOUND STORAGE

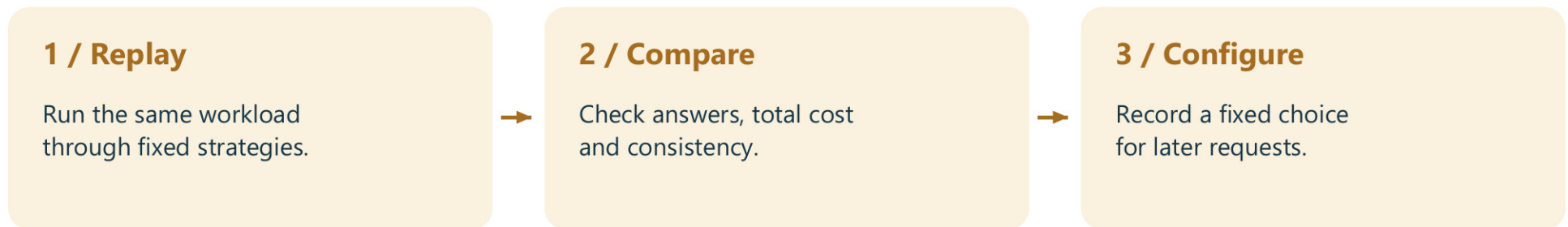
Evict least recently used entries.

RECHECK ACCESS

Saved computation never saves permission.

Adaptation is a measured decision

Keep the live path simple. Use recorded runs to decide what should change next.



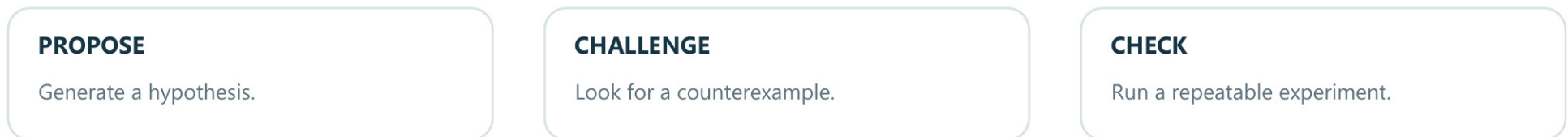
OPTIMIZATION OBJECTIVE

Saved compute > lookup + copying + profiling cost

Subject to matching results, current access checks and the storage budget.

Optional research swarm

Extension design / Not required



Standalone library: caching and access checks work without agents. The optional swarm would call this library through host-controlled interfaces; it cannot grant itself access or deploy its own configuration.

Reuse pays when preparation dominates

Local end-to-end execution, not production traffic. First parse included; all recorded runs retained.

Simple JSON reports

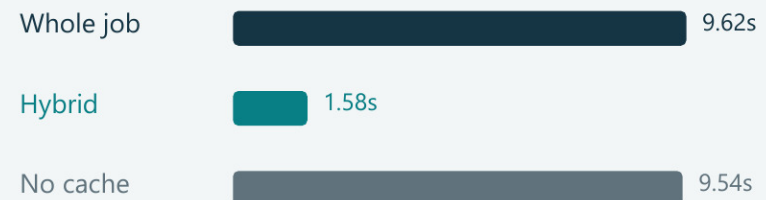
Original fixed workload mix



Whole-job caching wins

Real document / repeated queries

Parse and index an unchanged HTML page



84% less execution time with hybrid

Total measured seconds. Compare strategies within each panel.

Cold single query: hybrid was 7% slower than whole-job caching. The profiler keeps whole-job for this control and selects hybrid for repeated document queries.

Document source: [Python ast reference \(pinned HTML\)](#)

Data boundary: this prototype supports deterministic JSON structures with safe integers and rejects floats. Float/vector support is future adapter work, not a current feature. Reuse still must repay lookup, copying and amortized profiling cost; no universal crossover threshold is claimed.

Use it. Measure it. Keep the receipts.

A standalone library: host-controlled access checks and reusable computations. No swarm required.

```
python reproduce.py verify
```

```
python reproduce.py benchmark --workload document --output my-run
```

Requires Python 3.10+ and Node.js 18+. Verification and benchmarks use no network or paid APIs. Raw timings, source hashes and the full test inventory live in the kit.

Five lines to integrate

```
from thetabase import Cache
from pipeline import normalize_orders
cache = Cache(authorize=verify_user)
cache.register("normalize", "orders-v1", normalize_orders)
result = cache.call("normalize", batch, scope=user_id)
```

Your host supplies batch, user_id and verify_user(scope, stage), which returns Permit(allowed, revision). Initialize once; call the last line per request. The helper normalize_orders is included.

Research influences

Provenance-aware memory

Trace as State

Evaluation realism

Sources, hashes, raw measurements and the earlier experiments remain in the kit. These design influences do not establish novelty or production validation.